WEEK 4 / LESSON 5 / 50 MINUTES

L5

React Fundamentals

Every screen you use at Paytm, Zerodha, Cleartrip, Flipkart, and Uber is React. Today we cover the 20% of React that delivers 80% of daily work.

How many React apps did you use before lunch today?

WHAT'S INSIDE

WHY THIS LESSON MATTERS

Your AI is worthless until it has a UI.

NUMBER 1

20%

From real-world deployments.

NUMBER 2

80%

From real-world deployments.

SPEED

Now

A real reason this matters in industry today.

IMPACT

Yours

A real reason this matters in industry today.

LEARNING OUTCOMES

By the end of this lesson, you will be able to —

1. React's component + props + state mental model.
2. Build functional components with hooks.
3. Fetch + render API data with loading/error/success states.
4. Lift state up between sibling components.
5. Wire a dashboard to your Week-3 prediction API.

AGENDA

What we'll cover today

Short, sequenced parts. Each one builds on the last.

01

The mental model

Four words: Component · Props · State · JSX

02

Hooks — useState + useEffect

The two you use every day

03

Loading · Error · Success

The three states EVERY data view renders

04

Lift state up

When two siblings need the same data

01

PART 1 OF 4

The mental model

Four words: Component · Props · State · JSX

CONCEPT 1 OF 4

The mental model

Component — a function returning JSX.
Props — read-only arguments from parent.
State — values the component owns and changes.
JSX — HTML-like syntax inside JavaScript.
MEMORISE
C · P · S · JSX

Understand these four; everything else in React is syntactic.

DEFINITION

The mental model

Four words: Component · Props · State · JSX

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

CONCEPT 2 OF 4

Hooks — useState + useEffect

const [rows, setRows] = useState([]);
useEffect(() => fetch(url).then(r => r.json()).then(setRows), []);
Empty array = run once on mount.
Changing state re-renders the component.
PAIR
state · effect

90% of components need only these two hooks.

KEY POINTS

- const [rows, setRows] = useState([]);
- useEffect(() => fetch(url).then(r => r.json()).then(setRows), []);
- Empty array = run once on mount
- Changing state re-renders the component

CONCEPT 3 OF 4

Loading · Error · Success

Fetching — show a spinner or skeleton.
Failed — show a user-friendly error + retry.
Loaded — render the actual data.
Never show a blank screen while awaiting data.
CONTRACT
L · E · S

Ship all three branches from day one.

DEFINITION

Loading · Error · Success

The three states EVERY data view renders

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

CONCEPT 4 OF 4

Lift state up

Move the useState to their common parent.
Pass the value down as a prop.
Pass the setter down as a prop.
Children become controlled; parent is the source of truth.
MOVE
Up one level

Pattern repeats as apps grow. Master once.

KEY POINTS

- Move the useState to their common parent
- Pass the value down as a prop
- Pass the setter down as a prop
- Children become controlled; parent is the source of truth

02

PART 2 OF 4

Engage and reflect

Pause, talk, and connect what you just learned to your own work.

✦ THINK · PAIR · SHARE

ACTIVITY

Build a PriceCard component

PROMPT

Apply the four concepts above to one real example from your own week.

HOW TO RUN IT

1.

2.

3. Paste component code; class critiques naming + shape.

4. Reveal: instructor confirms the canonical answer.

REFLECT & APPLY

Three questions before you close this lesson

Spend 60 seconds on each. Write the answers down — no scrolling, no shortcuts.

Q1 *Which React concept scared you before this lecture?*

Q2 *Where have you seen "state lifted too high" (prop-drilling)?*

Q3 *What is the first dashboard you will build next week?*

03

PART 3 OF 4

Knowledge check

Three short questions. One minute each. Honest answers only.

KNOWLEDGE CHECK · QUESTION 1 OF 3

Props are:

A Mutable

B Read-only

C Async

D Encrypted

ANSWER · QUESTION 1 OF 3

Props are:

B

CORRECT ANSWER

Read-only

WHY

Props flow from parent → child and must not be mutated by the child

KNOWLEDGE CHECK · QUESTION 2 OF 3

useEffect with []:

A Every render

B Once on mount

C On unmount

D Never

ANSWER · QUESTION 2 OF 3

useEffect with []:

B

CORRECT ANSWER

Once on mount

WHY

Empty deps = run exactly once after first render

KNOWLEDGE CHECK · QUESTION 3 OF 3

Three states every data view renders:

A Pending/Active/Done

B Loading/Error/Success

C Red/Green/Blue

D Past/Present/Future

ANSWER · QUESTION 3 OF 3

Three states every data view renders:

B

CORRECT ANSWER

Loading/Error/Success

WHY

Loading, Error, Success — always all three

04

PART 4 OF 4

Apply and close

One thing to remember. One thing to ship this week.

SUMMARY

Five sentences to remember

If you remember nothing else from this lesson, remember these five lines.

1. Pick the right tool — never the loudest one.
2. Practice the framework on a real example.
3. Watch for the three classic pitfalls.
4. Document your decision in one sentence.
5. Carry the discipline forward to the next lesson.

WORKED EXAMPLE

How this lesson plays out in one real Indian context

THE SCENARIO

Component — a function returning JSX.
Props — read-only arguments from parent.
State — values the component owns and changes.
JSX — HTML-like syntax inside JavaScript.
MEMORISE
C · P · S · JSX

Understand these four; everything else in React is syntactic.

THE TAKEAWAY

"Components own state. Props flow down. Events flow up. 80% of React."

WHAT TO NOTICE

1. The mental model
2. Hooks — useState + useEffect
3. Loading · Error · Success
4. Lift state up

APPLY IT → ASSIGNMENT

Mini assignment — deploy a dashboard

Build a React dashboard on your W3 prediction API. Deploy to Vercel.

DELIVERABLES

- Form + result card + history table.
- Loading / error / success branches.
- vercel deploy → public URL.
- Submit URL on the forum.

WHAT 'GOOD' LOOKS LIKE

Deliverable: Public Vercel URL.

ONE THING TO REMEMBER

> *"Components own state. Props flow down. Events flow up. 80% of React."*

— AI Learning Hub

END OF LESSON 4

What you can now do

Each one of these is now part of your working vocabulary.

1. Pick the right tool — never the loudest one.
2. Practice the framework on a real example.
3. Watch for the three classic pitfalls.
4. Document your decision in one sentence.
5. Carry the discipline forward to the next lesson.

NEXT → Lesson 6 — Process Orchestrati on. Sequencing models and bots across a pipeline.